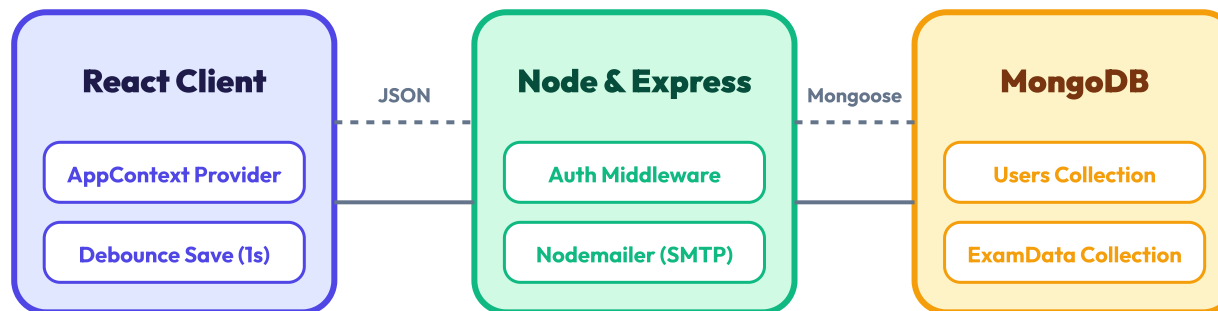


Catalyst Study Tracker .

MERN Stack Architecture, System Workflows, Debugging History, and Concept Documentation

1. System Architecture & High-Level Workflow

Catalyst is designed as a highly responsive, modern, single-page MERN stack application. It coordinates an asynchronous client workspace using a debounced data pipeline that writes directly to MongoDB, enabling rapid interface states with secure database persistence.



Complete Project Directory Tree

```
gambit/
├── backend/
│   ├── config/
│   │   └── db.js          # Mongoose MongoDB Connection Connector
│   ├── middleware/
│   │   └── auth.js        # JWT Header Verification Token Validator
│   ├── models/
│   │   ├── User.js        # User profile, custom exam definitions, & reset codes Schema
│   │   ├── ExamData.js    # Streak, XP, syllabus progress & PYQs Schema
│   │   └── Snapshot.js    # User configuration backups Schema
│   ├── routes/
│   │   ├── auth.js        # Signup, Login, Profile, Custom Exams, & OTP endpoints
│   │   ├── exam.js        # Syncs details, loads templates, & updates scores
│   │   └── snapshot.js    # REST points capture and restoration APIs
│   ├── package.json       # Backend Dependencies configuration
│   ├── server.js          # Entry node server configuration
│   └── setup-email.js     # CLI automatic email credential configurator
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Dashboard.jsx # Main interface widget view showing overall metrics
│   │   │   ├── ExamSetup.jsx # Prompts selection of templates or creation of custom ones
│   │   │   ├── ExamSwitcher.jsx # Swaps currently active workspace context
│   │   │   ├── Header.jsx    # Displays streak index, active score (XP) points, and profiles
│   │   │   ├── Login.jsx     # Sign up, Sign in, and two-step OTP verification form
│   │   │   ├── Placeholders.jsx # Spaced repetition revision list & PYQ analysis tabs
│   │   │   ├── Planner.jsx   # Daily checklist scheduler
│   │   │   └── SyllabusTracker.jsx # Subjects, topics, and completion status matrix
│   │   └── context/
```

```
| | | └─ AppContext.jsx    # React Context provider executing auto-save loops
| | └─ data/
| |   └─ examsData.jsx    # Default static syllabus matrices & trends template
| |   └─ App.jsx          # Routes setup & Protected routes middleware
| |   └─ main.jsx         # React mounting file
| └─ vite.config.js       # Directs /api calls to backend server
└─ setup-email.bat        # Double-clickable utility script for setup
```

2. Backend & Frontend Code Breakdown

Backend: User Model with Verification Fields (models/User.js)

The user model maps authentication records directly to MongoDB. We have extended the schema to support temporary verification values `resetCode` and `resetCodeExpires` which hold the OTP credentials during password recovery states:

```
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true },
  avatarType: { type: String, default: 'emoji' },
  emoji: { type: String, default: '👤' },
  picture: { type: String, default: '' },
  enrolledExams: { type: [String], default: ['UPSC CSE'] },
  selectedExam: { type: String, default: 'UPSC CSE' },
  customExams: { type: Array, default: [] },
  resetCode: { type: String, default: null },
  resetCodeExpires: { type: Date, default: null },
  createdAt: { type: Date, default: Date.now }
});
```

Backend: Authentication & OTP verification endpoints (routes/auth.js)

The endpoints handle password reset requests securely. `/forgot-password` generates a random 6-digit verification code, stores it in MongoDB with a 15-minute expiration, and uses `nodemailer` to dispatch the email. It automatically falls back to an output simulator log if no SMTP variables are set in `.env`.

```
// POST /api/auth/forgot-password - Generate and send OTP reset code
router.post('/forgot-password', async (req, res) => {
  try {
    const { email } = req.body;
    const user = await User.findOne({ email });
    if (!user) return res.status(404).json({ msg: 'User with this email does not exist' });
    // Generate 6-digit random code and set 15-minute expiration
    const resetCode = Math.floor(100000 + Math.random() * 900000).toString();
    user.resetCode = resetCode;
    user.resetCodeExpires = Date.now() + 15 * 60 * 1000;
    await user.save();
    const resetHtml = `

## Reset Your Catalyst Password



Your 6-digit OTP code is: ${resetCode}

`;
    const mailStatus = await sendMail(email, 'Your Catalyst Password Reset OTP Code 🗝️', resetHtml);

    if (mailStatus && mailStatus.simulated) {
      return res.json({ msg: 'OTP generated (SIMULATION)', simulated: true, code: resetCode });
    }
    res.json({ msg: 'Verification OTP sent to email' });
  } catch (err) {
    res.status(500).send('Server error');
  }
});
```

Frontend: State Auto-Save & Sync Loop (context/AppContext.jsx)

The state manager utilizes a debouncing mechanism to throttle modifications to the remote database, avoiding overloading the Node.js event loop with continuous database writes:

```

// Synchronizes local UI state changes to MongoDB with a 1-second debounce delay
useEffect(() => {
  if (isAuthenticated && selectedExam && selectedExam === currentExamRef.current && !isLoadingExam) {
    const delayDebounceFn = setTimeout(async () => {
      try {
        await apiFetch(`/api/exam/${selectedExam}`, {
          method: 'PUT',
          body: JSON.stringify({
            syllabusData,
            notes,
            shortNotes,
            mistakes,
            revisions,
            mockTests,
            dailyStudy,
            todos,
            dDay,
            dDayPhases,
            targetScore,
            streak,
            points,
            lastLoginDate
          })
        });
      } catch (err) {
        console.error("Autosave synchronization failed", err);
      }
    }, 1000);
    return () => clearTimeout(delayDebounceFn);
  }
}, [syllabusData, notes, shortNotes, mistakes, revisions, mockTests, dailyStudy, todos, dDay, dDayPhases,

```

```
targetScore, streak, points, lastLoginDate]]);
```

3. Problems Faced & How They Were Solved

Problem Encountered	Root Cause	Professional Resolution
White Screen of Death (Crash) on Notes/PYQs page load	Accessing <code>s.topics</code> or <code>subject.topics</code> via <code>.flatMap()</code> or <code>.forEach()</code> in components when no topics array is defined (e.g. for a newly added custom exam or new subject).	Secured every <code>.topics</code> invocation across all component files with default fallbacks, e.g. <code>(subject.topics [])</code> , ensuring the app never crashes on empty states.
ReferenceError: ExamData is not defined in backend router	The <code>ExamData</code> mongoose model was used inside <code>/me</code> GET route to aggregate cross-exam target scores, but it was not imported at the top of <code>backend/routes/auth.js</code> .	Imported the model explicitly: <code>const ExamData = require(' ../models/ExamData')</code> at line 5 of <code>routes/auth.js</code> .
Gmail delivery failing or requiring hardcoded credentials	Unauthenticated relay services are blocked by Gmail's anti-spam rules. Forcing the user to manually edit <code>.env</code> creates configuration complexity.	Installed <code>nodemailer</code> , set up standard Gmail SMTP routing, and wrote an automatic setup script (<code>setup-email.bat</code>) which prompts users inside their terminal to quickly configure the details.

Problem Encountered	Root Cause	Professional Resolution
Autosave overloading database on rapid keypresses	Triggering a network PUT request to sync the database on every single keystroke lags the frontend and overwhelms MongoDB connection pools.	Implemented a 1-second debounce timeout inside the React <code>useEffect</code> sync hook. Any new keypress clears the previous timeout and restarts the 1-second delay timer.

4. Production Scaling Roadmap

To transition this MERN application from a local development workspace to supporting millions of concurrent users, we would apply the following cloud scale enhancements:

- **API Clustering & PM2:** Deploy the Node.js API servers to cloud containers running under a PM2 Process Cluster to utilize multi-core server processors via reverse-proxy load balancing (such as Nginx).
- **MongoDB Atlas sharding:** Migrate the local database to MongoDB Atlas with sharded clusters, using the user's `userId` as the shard key. This distributes user exam profiles and snapshot files across distinct database clusters.
- **Caching Layer (Redis):** Use Redis in front of the API endpoints to cache user snapshots and static exam syllabi configurations. This reduces database queries for frequently read operations.
- **SMTP Delivery Relays:** Swap out Gmail's daily limits with transactional cloud SMTP relays (such as Amazon SES, Brevo, or SendGrid) to securely send high volumes of emails and verification codes.

5. MERN Stack Interview Questions & Answers

Q: Explain the React Context API. When and why should we use it over Redux?

React Context is a built-in feature that enables passing state down a component tree without manually threading props (prop drilling) at each nested level. Context is ideal for application-level metadata (such as authentication status, user profiles, theme styles, or shared exam setups). Redux is preferred for highly complex state transitions with strict tracking, action middleware pipelines, or frequent state updates, whereas Context is lightweight and requires no external packages.

Q: What is debouncing, and how did we implement it in the Catalyst sync loop?

Debouncing is a software pattern that limits the execution of a function until a certain amount of time has elapsed since the last request was made. In Catalyst, we implement it using a React `useEffect` timer. When state variables change, we start a 1-second `setTimeout` to write the data to the backend. If another change occurs within that 1 second, the cleanup function triggers `clearTimeout`, cancelling the previous write. This ensures we only perform a network database write once the user stops typing for at least 1 second.

Q: Why did we run into a `ReferenceError` in node, and how can we prevent it during import cycles?

A `ReferenceError` occurs when the JavaScript engine encounters a variable name it cannot resolve in any active scope. In our case, the router was using the `ExamData` model to query MongoDB, but the import statement was missing from that route file. To prevent this, developers use strict static analysis/linting tools (like ESLint) during compilation, and check for module cycle dependency leaks (circular dependencies) where two modules require each other, causing unresolved objects.

Q: Explain Node.js event-driven, non-blocking asynchronous architecture.

Node.js runs on a single main thread using an Event Loop. When an asynchronous operation (like a MongoDB query, file read, or network mail transmission) is triggered, Node hands the task over to the operating system's system threads or thread pool (via libuv). The main execution thread continues handling other incoming HTTP requests. When the OS finishes the asynchronous task, it places the callback into the event queue, and the Event Loop runs it once the execution stack is clear. This makes Node highly efficient at managing thousands of concurrent I/O connections.

Q: How does JWT authentication work, and how is it secured?

A JSON Web Token (JWT) is a cryptographically signed payload split into three sections: Header, Payload, and Signature. During login, the server generates a token signed using a private key (`JWT_SECRET`). The client stores this token in browser local storage or secure cookies. For every subsequent request, the client includes the token in the **Authorization: Bearer [token]** header. An Express middleware validates the signature. It is secured by keeping the secret key hidden on the server, enforcing expiration times, and using HTTPS to prevent token interception.